ROS 2 with Kubernetes and KubeEdge

by *Tomoya Fujita* / ***fujitatomoya@github***

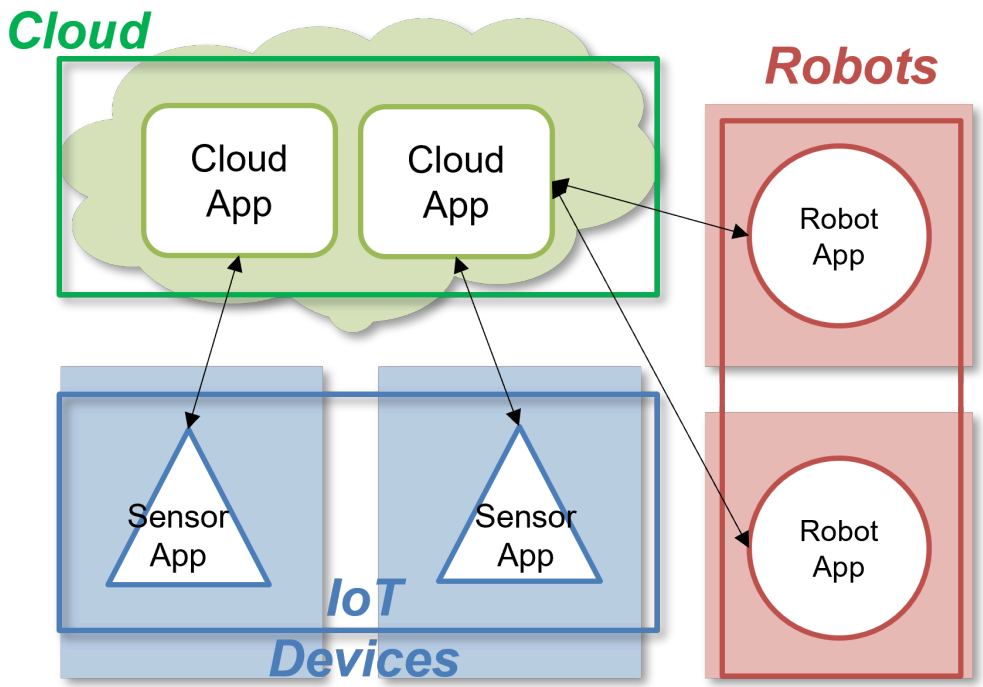
Agenda

- Background, Motivation and Goal
- Why Kubernetes (and KubeEdge) for Robotics
- The `ros_k8s` Repository: What is Verified Today
- Networking: Cilium, Multicast, Hubble and Encryption
- ROS 2 Deployment Patterns
- KubeEdge: Cloud / Edge Coordination
- Roadmap / Open Topics
- References

by *Tomoya Fujita* / ***fujitatomoya@github***

Background / Problems

- Broad use cases (factory, logistics, entertainment, rescue, car, drone...)
- ROS 2 is a distributed and connected system (DDS, multicast discovery)
- Collaborative and orchestrated applications across many devices
- AI / ML Ops and application lifecycle at the edge
- Hardware acceleration and platform dependencies
- Everything tends to be **statically configured and integrated**

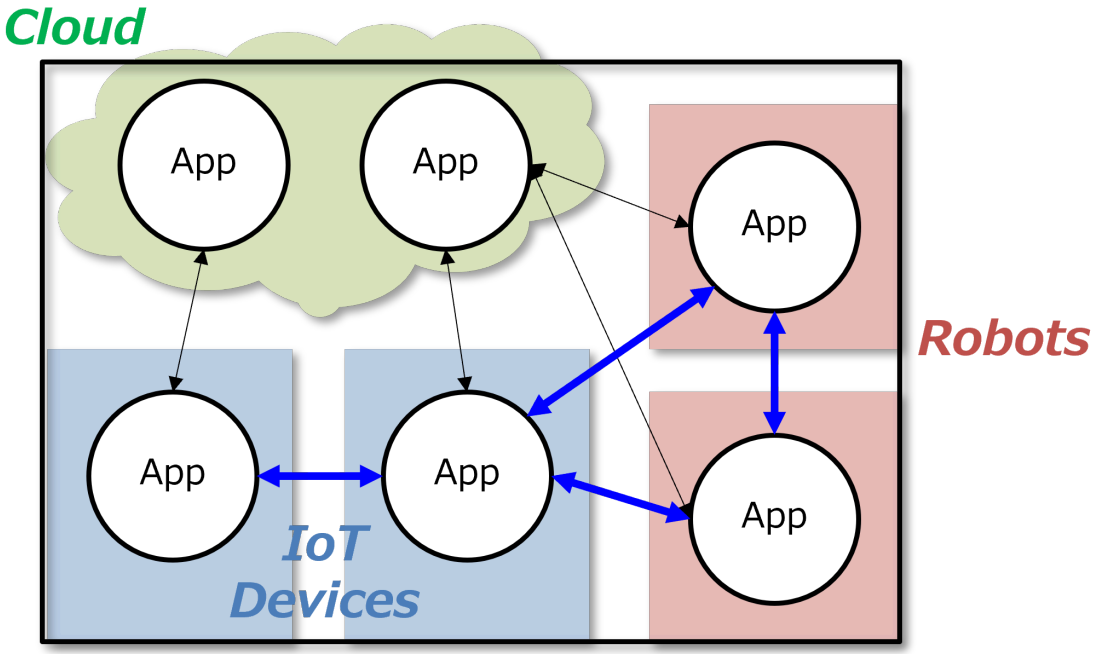


Complicated
&
Single System
&
Specified

What's the pain?

- Different architecture for cloud and edge device.
- Take time and effort to setup environment and run application.
- IoT Device is really statistic implementation.

Simple/Common
&
Distributed System
&
Platform Agnostic



What we want is…

- Common base architecture for everyone and everywhere.
- Application can be deployed on anywhere.
- Eco-system for Application.

Goal / Requirements

- Flexible and configurable application deployment
- Application agnostic network, device and storage abstraction
- Application modularity / platform agnostic infrastructure
- Sustainability / efficiency on resource constrained devices
- System global observability / network policy / security
- **Everyone can try ROS 2 with Kubernetes and KubeEdge**
  - that is the motivation of fujitatomoya/ros_k8s

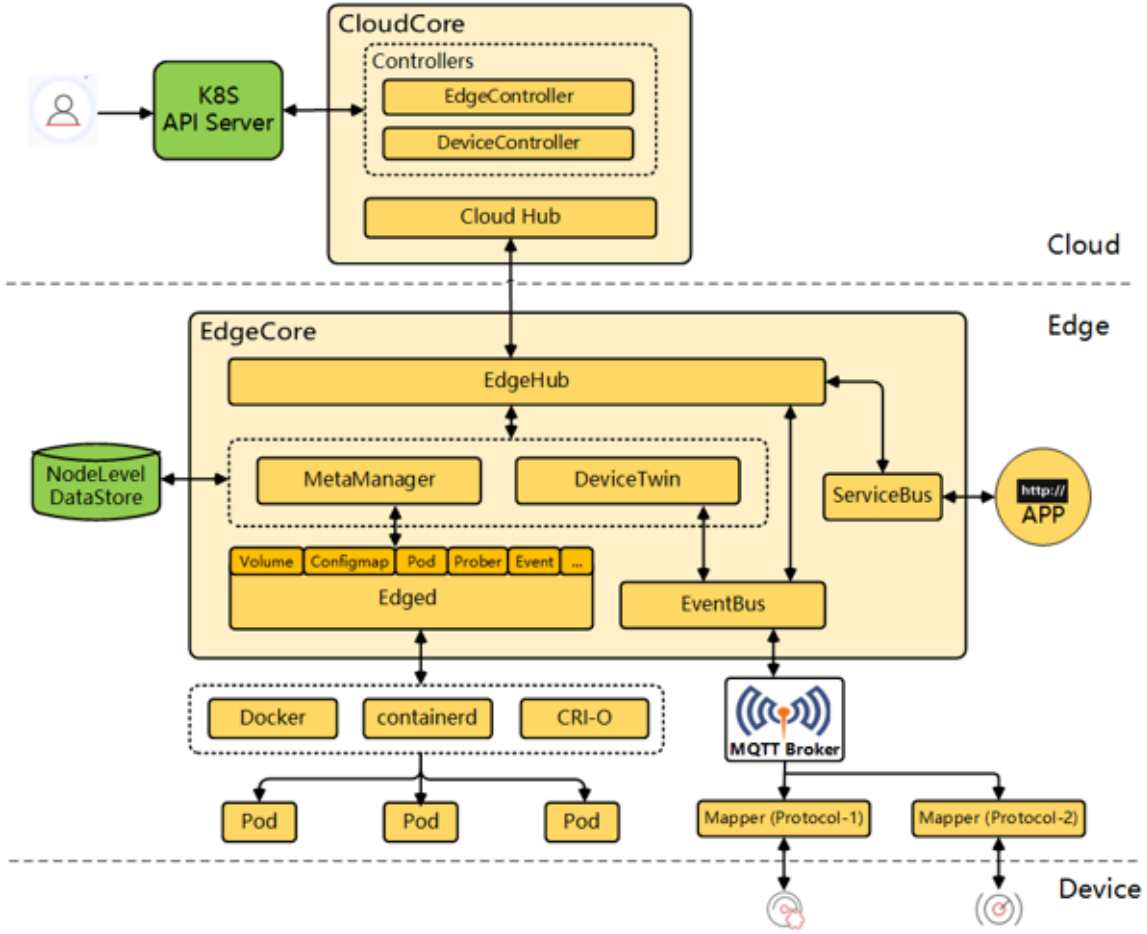
Kubernetes

- Application deployment and orchestration
- Device capability and label control (`nodeSelector`)
- Auto-scaling and self-healing (`restartPolicy`)
- Rolling update / rollback, canary test
- Role Based Access Control
- Device Plugin / Container Device Interface (CDI)
- Container Network Interface (CNI)
- Observability / Network Policy



KubeEdge

- Cloud-Edge coordination over the Internet (NAT friendly tunnel)
- Edge autonomy (works while offline)
- Kubernetes-native: edge nodes are just nodes
- Resource efficient (`edgecore` replaces `kubelet`)
- CNCF graduated project, SIG Robotics



by *Tomoya Fujita* / ***fujitatomoya@github***

The `ros_k8s` Repository

- Tutorials to run ROS / ROS 2 on Kubernetes and KubeEdge, end to end
  - install packages, build multi-arch images (amd64 / aarch64)
  - physical cluster (`kubeadm`) or virtual cluster (`kind`)
  - CNI (Cilium), multicast, observability, encryption, Headlamp
  - ROS 2 deployment samples from basic to intermediate
  - KubeEdge cloudcore / edgecore and resource management
- Container images: `tomoyafujita/ros:{rolling,jazzy,noetic}`
- Everything is re-verified in 2026 on Ubuntu 24.04 LTS

Verified Versions (October 2026)

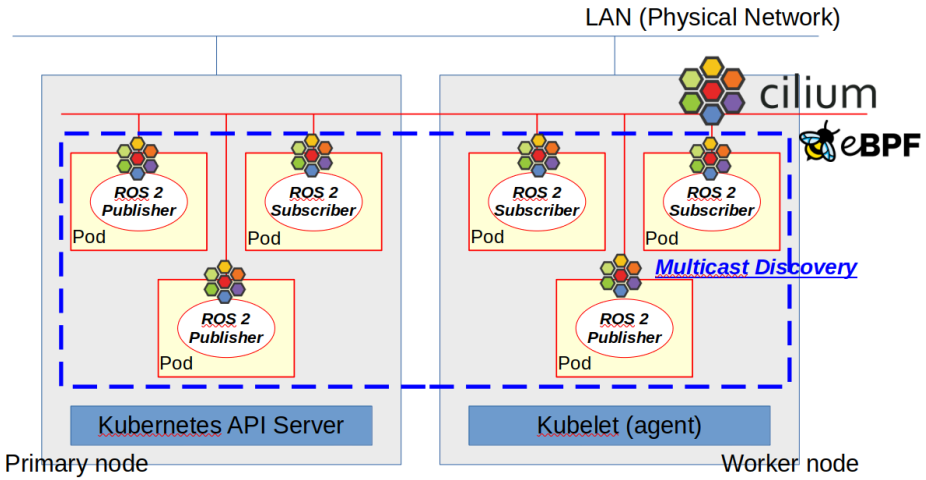
Component	Version
Ubuntu	24.04 LTS (Noble), amd64 / aarch64 (Raspberry Pi 4/5)
Kubernetes	v1.37
KubeEdge	v1.23.1 (verified with Kubernetes v1.30 - v1.32)
kind	v0.33.0 (`kindest/node:v1.37.0`)
Cilium	v1.20.x (cilium-cli v0.20.x)
containerd	2.x (Docker apt repository)
ROS 2	Jazzy (LTS, Ubuntu 24.04) / Rolling (Ubuntu 26.04 image)
ROS 1	Noetic (EOL May 2025, kept for reference)

Cluster Options: Physical or Virtual

- **Physical host cluster** (`kubeadm init` / `kubeadm join`)
  - the practical environment for robots, sensors and accelerators
  - `containerd` with `SystemdCgroup = true`, swap disabled
- **Virtual cluster with kind** (Kubernetes IN Docker)
  - `kind create cluster --config=./yaml/kind-multiple-node.yaml`
  - 3 virtual nodes on one host, `disableDefaultCNI: true` to deploy Cilium
  - `kind load docker-image` to use locally built images
  - everything else is the same `kubectl` experience
- **Headlamp** (Kubernetes SIG UI) replaces the retired Kubernetes Dashboard

CNI Matters for ROS 2 / DDS

- DDS discovers participants via **UDP multicast** (`239.255.0.1`)
- Flannel, Calico, kindnet: no multicast across nodes
- WeaveNet used to be the answer, now **end of life**
- **Cilium** is the primary CNI in this tutorial
  - eBPF based multicast (**beta**)
  - tunnel (`vxlan`) routing mode
  - kernel 5.10+ (amd64) / 6.0+ (arm64)



Enable Cilium Multicast for DDS Discovery

```
> cilium install --version 1.20.2
> cilium status --wait

# enable multicast (restarts cilium-agent pods)
> cilium config set multicast-enabled true

# make all nodes join the ROS 2 / DDS well-known group at once
> cilium multicast add --group-ip 239.255.0.1
> cilium multicast list group
> cilium multicast list subscriber --all
```

- One command for the whole cluster, no per-node `cilium-dbg bpf multicast ...`
- Discovery across nodes just works, no change in the application container

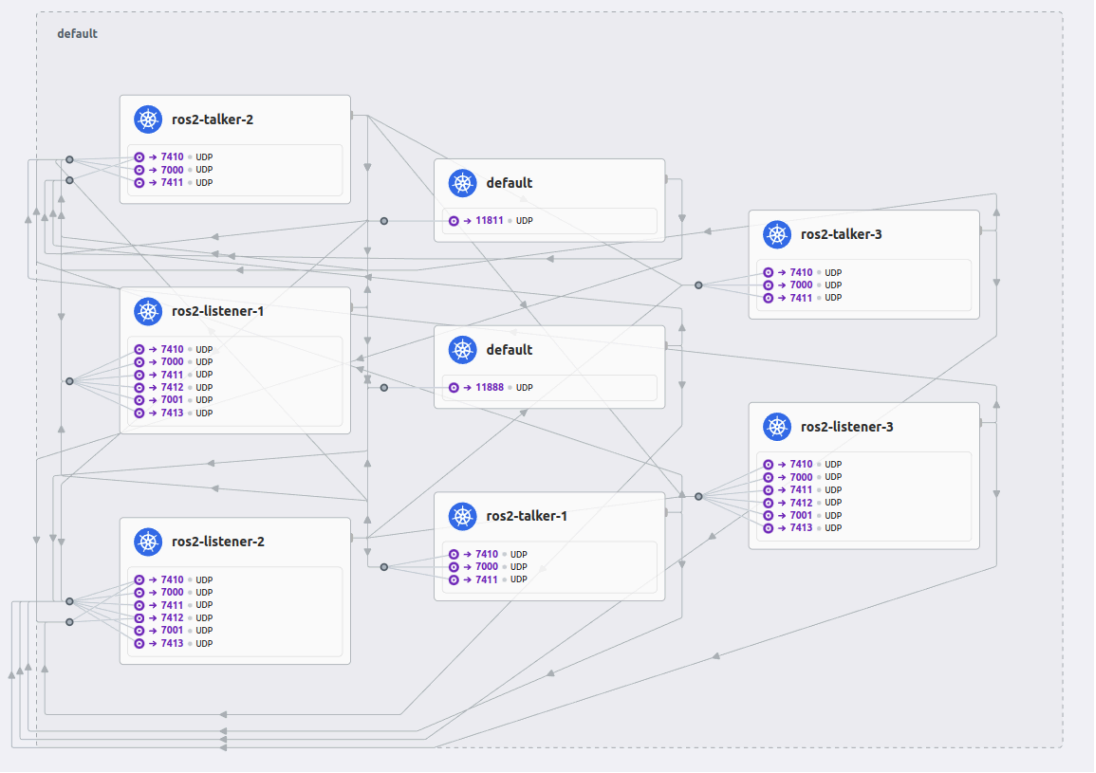
by *Tomoya Fujita* / ***fujitatomoya@github***

Hubble Observability

- eBPF based network flow visibility for ROS 2 pods
- No sidecar, no change in the application

```
> cilium hubble enable --ui
> cilium hubble ui
# http://localhost:12000
```

- Service map shows who talks to whom (DDS discovery, topics)
- Hubble CLI for flow level debugging



by *Tomoya Fujita* / ***fujitatomoya@github***

Transparent Encryption: IPsec / WireGuard

- Cilium encrypts node-to-node pod traffic with IPsec or WireGuard

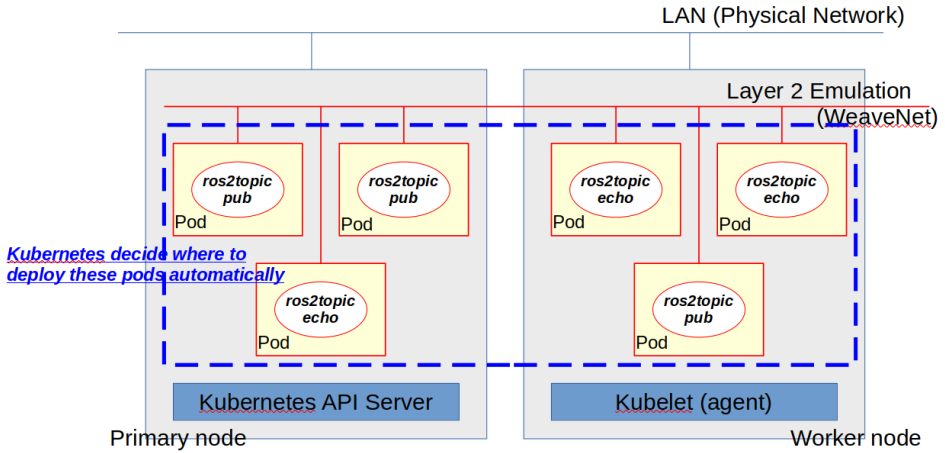
```
> kubectl create -n kube-system secret generic cilium-ipsec-keys \
    --from-literal=keys="3+ rfc4106(gcm(aes)) $PSK 128"
> cilium install --version 1.20.2 \
    --set encryption.enabled=true --set encryption.type=ipsec
```

- Per-tunnel keys (`+`) recommended, global keys deprecated since v1.16
- **Multicast does not work together with IPsec encryption**
  - tracking issue: cilium/cilium#29471
  - choose: multicast discovery **or** IPsec, or go unicast (discovery server)

Pattern 1: Simple Distributed System

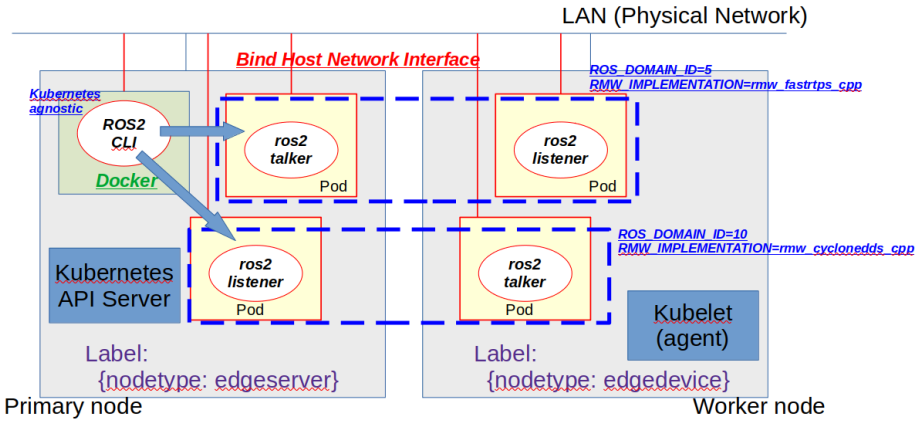
- Talkers and listeners as `Deployment`, no `nodeSelector`
- Kubernetes decides where to run, restarts on failure
- Cilium multicast delivers DDS discovery to all nodes
- "I do not care where, but it must be running somewhere"

```
> kubectl apply -f ./yaml/ros2-sample.yaml
> kubectl logs deployment/ros2-listener-3
data: Hello, I am talker-3
```



Pattern 2: Localhost Only / Logical Partition

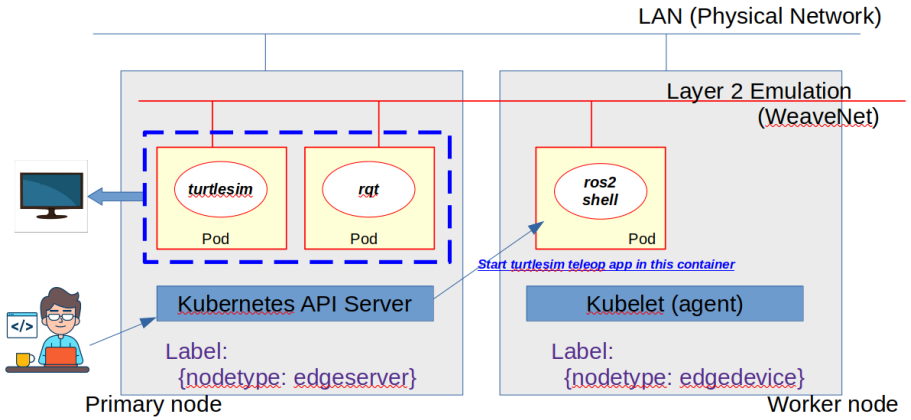
- **Localhost only**: `DaemonSet` per node
  - `ROS_AUTOMATIC_DISCOVERY_RANGE=LOCALHOST`
  - one virtual localhost per pod, multi-user on one node
  - `ROS_LOCALHOST_ONLY` is deprecated since Iron
- **Logical partition**: node labels + `ConfigMap`
  - `ROS_DOMAIN_ID` / `RMW_IMPLEMENTATION` injected as env
  - Fast DDS, Cyclone DDS, Connext configurations
  - configuration is separated from the container image



Pattern 3: GUI Display Access

- Label the node that has a display: `nodetype=edgeserver`
- Mount `/tmp/.X11-unix`, set `DISPLAY`, `xhost +local:root`
- `turtlesim` and `rqt` run on the display node, `teleop_key` on `edgedevice`
- Ubuntu 24.04 Wayland session works via `Xwayland`

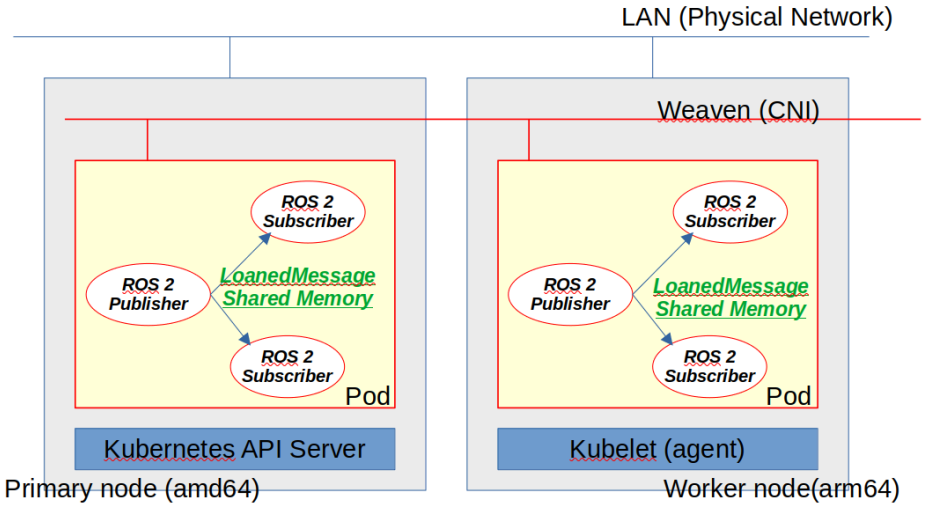
```
> kubectl apply -f ./yaml/ros2-turtlesim.yaml
> kubectl exec -it <teleop-pod> -- ros2 run turtlesim turtle_teleop_key
```



Pattern 4: Zero Copy Data Sharing

- ROS 2 `LoanedMessage` + Fast DDS Data Sharing Delivery
- True zero copy between containers in the **same pod**
- Bind one `/dev/shm` tmpfs to all containers in the pod
- No daemon process required (Fast DDS)

```
> kubectl apply -f ./yaml/ros2-data-sharing.yaml
> kubectl exec <pod> -c ros2-listener-1 -- ls /dev/shm
fastdds_1db5a9b5dadec5a6
```

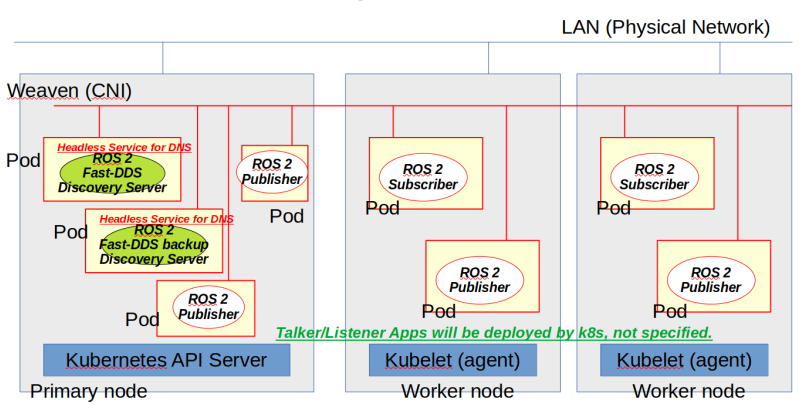


Pattern 5: Fast DDS Discovery Server

- Centralized discovery, **no multicast required** (works with Flannel)
- Primary / secondary: `fastdds discovery -i <id> -p <port>`
- **Headless Service** (`clusterIP: None`) gives a stable DNS name

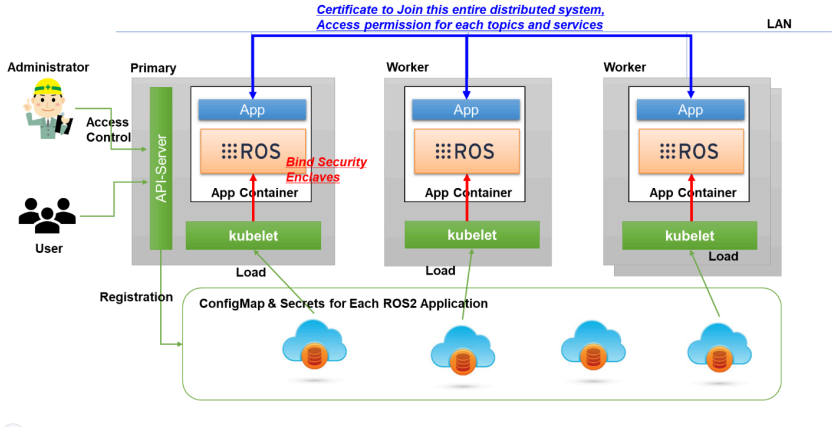
```
- name: ROS_DISCOVERY_SERVER
  value: "primary-discovery-server:11811;secondary-discovery-server:11888"
```

- Resolved by cluster DNS at startup, no static IP in the application
- `ROS_SUPER_CLIENT=true` for `ros2cli` introspection



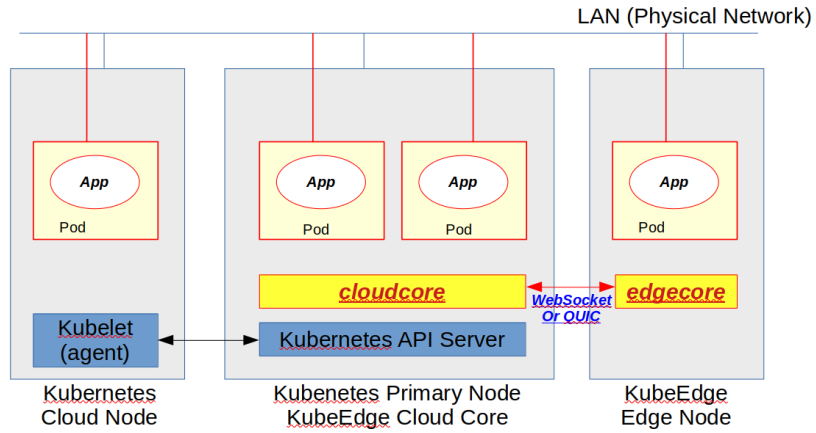
Pattern 6: SROS2 Enclaves with ConfigMap

- Admin generates enclaves: `ros2 security create_keystore` / `create_enclave`
- `cp -L` and `kubectl create configmap --from-file` (no symlinks in ConfigMap)
- Env ConfigMap: `ROS_SECURITY_KEYSTORE` / `ROS_SECURITY_ENABLE` / `ROS_SECURITY_STRATEGY=Enforce`
- Kubernetes binds the right enclave wherever the pod starts
- ConfigMap is not encrypted: use **Secret** for `key.pem` in production



KubeEdge: cloudcore / edgecore

- Cloud: Kubernetes API server + CNI, then `keadm init --advertise-address=<IP>`
  - ports `10000` / `10002` must be reachable from the edge
- Edge: `containerd` only, **no kubelet**, `keadm join`
  - `--kubeedge-version=v1.23.1 --cgroupdriver=systemd`
- Edge node shows up with role `agent,edge` in `kubectl get nodes`
- `edgecore` does not deploy CNI pods: ROS 2 pods on the edge use `hostNetwork: true`
- `kubectl logs/exec` on the edge needs `cloudStream` / `edgeStream`



Cilium with KubeEdge

- KubeEdge docs: Cilium (KubeEdge v1.16+)
  - `cilium install --set encryption.type=wireguard`
  - `configure_cilium.sh cloudcore` / `configure_cilium.sh edgecore`
  - cilium-agent API calls go through `MetaManager` on the edge
  - edge pods get Cilium IPs and talk to cloud pods over WireGuard
- Cilium + eBPF Day NA 2024: Applying Cilium at Edge with KubeEdge
  - Cilium + WireGuard on every cloud and edge node = one data plane
  - open: WireGuard topologies (hub-and-spoke, site-to-site), lighter footprint, scale to millions of devices
- Not yet in this tutorial: Cilium (multicast / WireGuard) on the KubeEdge edge node with ROS 2

KubeEdge Hold and Release

- Problem: an uncontrolled rolling update can stop a moving robot
- Hold and Release (KubeEdge v1.22+): the **edge decides when** to upgrade
- Pod template annotation `edge.kubeedge.io/hold-upgrade: "true"`
  - `edged` caches the new pod, reports `HeldUpgrade` condition
  - held upgrade survives edge node restart (MetaManager)
- Release from the edge node (MetaServer required)

```
> keadm ctl unhold-upgrade node <node-name>
```

- Pods, Deployments, StatefulSets, DaemonSets are supported
- Natural fit for robots: upgrade when docked, not while driving

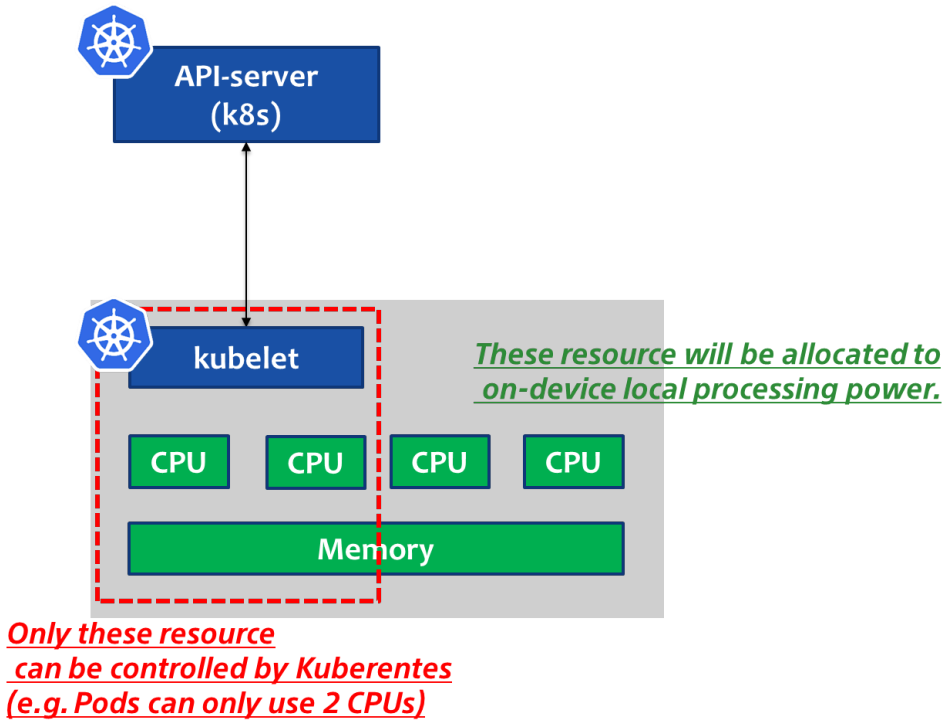
by *Tomoya Fujita* / ***fujitatomoya@github***

Resource Reservation for Edge

- Robots need CPU / memory for the host system (drivers, real-time)
- `kubelet` Node Allocatable in `/var/lib/kubelet/config.yaml`

```
reservedSystemCPUs: "2-3"
systemReserved:
  memory: 4Gi
evictionHard:
  memory.available: "5%"
```

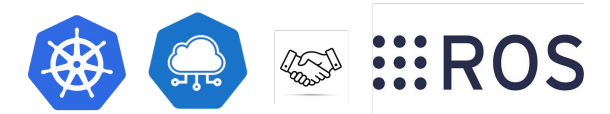
- Raspberry Pi 4: allocatable 4 -> 2 CPUs, 8 -> 4 GiB
- Pod `requests` / `limits` on the application side



by *Tomoya Fujita* / ***fujitatomoya@github***

Still Missing

- Cilium (multicast + WireGuard) on KubeEdge edge nodes, verified with ROS 2
- Multicast together with encryption
- Hardware acceleration: GPU / device plugin / CDI samples
- Far-edge devices: micro-ROS, WasmEdge
- Edge-aware rollout: hold and release, in-place update
- **Feedback and use cases are welcome**



Kubernetes IoT Edge WG

KubeEdge SIG Robotics

ROS Kubernetes Tutorials

by *Tomoya Fujita* / ***fujitatomoya@github***

References

- Repository: github.com/fujitatomoya/ros_k8s
- ROSCon 2023: Slides / Talk
- ROS By the Bay 2023: Slide Deck
- Cilium + eBPF Day NA 2024: Applying Cilium at Edge with KubeEdge
- KubeEdge: Documentation / Cilium / Hold and Release / KubeEdge meets Cilium
- Cilium: Documentation / Multicast / IPsec
- Kubernetes / kind / Headlamp
- ROS 2 Jazzy / ROS 2 Rolling

Questions?

github.com/fujitatomoya/ros_k8s

by *Tomoya Fujita* / ***fujitatomoya@github***